



Kubernetes — The Complete Notes

A deep, visual, and practical guide to Kubernetes — from "why do I need an orchestrator" all the way to Pods, Deployments, Services, Ingress, ConfigMaps, Secrets, Helm, and production best practices.



Table of Contents

1. [Why Kubernetes Exists](#)
2. [Docker vs Kubernetes](#)
3. [Kubernetes Architecture](#)
4. [Core Objects: Pods, ReplicaSets, Deployments](#)
5. [Services & Networking](#)
6. [Ingress](#)
7. [ConfigMaps & Secrets](#)
8. [Volumes & Persistent Storage](#)
9. [Namespaces, Labels & Selectors](#)
10. [StatefulSets, DaemonSets, Jobs & CronJobs](#)
11. [Scheduling, Probes & Autoscaling](#)
12. [RBAC & Security](#)
13. [Helm — The Package Manager](#)
14. [kubectl Cheat Sheet](#)
15. [Real-World Example](#)
16. [Troubleshooting](#)

1. Why Kubernetes Exists

"Okay, I have 200 containers across 12 servers. Now what?" — Every team that outgrew Docker Compose.

Docker solves *packaging*. Kubernetes solves what happens **after** you have hundreds of containers running across many machines:

- A container crashes — **who restarts it?**
- Traffic spikes — **who adds more replicas?**
- A node dies — **who reschedules its workloads?**
- You ship a new version — **who rolls it out without downtime?**
- One service needs to find another — **how do they discover each other?**

Kubernetes (a.k.a. **K8s** — 8 letters between K and s) is the **container orchestrator** that does all of that for you, automatically, declaratively.

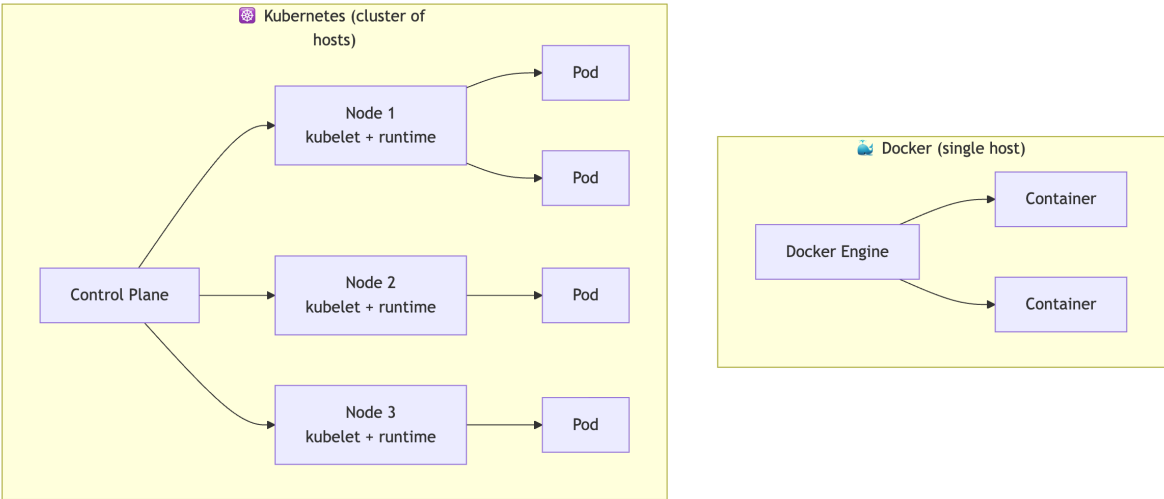
Key benefits

Benefit	What it means
Self-healing	Restarts failed containers, reschedules dead nodes
Declarative	You describe <i>desired state</i> ; K8s makes it real
Scalable	Horizontal autoscaling on CPU/memory/custom metrics
Portable	Runs on any cloud, on-prem, or your laptop

Rolling updates	Zero-downtime deploys + instant rollbacks
Service discovery	Built-in DNS + load balancing between Pods

2. Docker vs Kubernetes

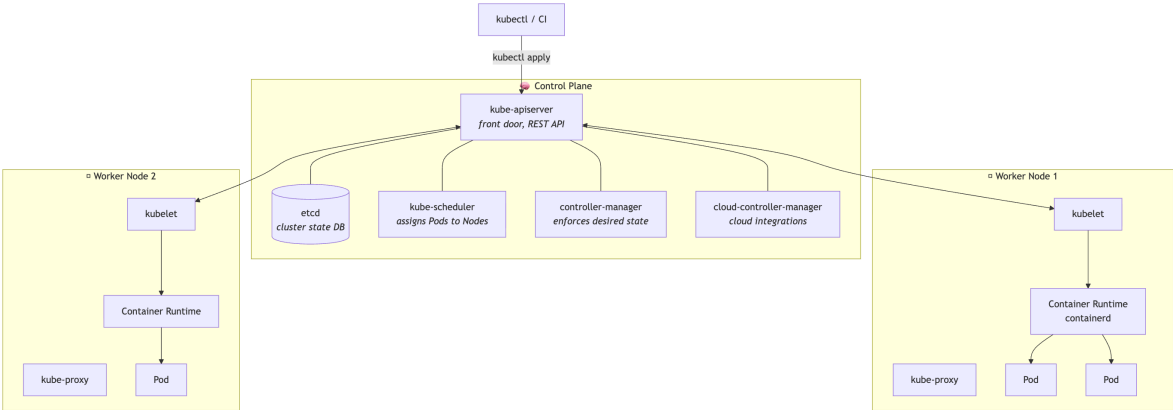
They're **not competitors** — they work together. Docker builds and runs single containers. Kubernetes orchestrates many containers across many machines.



Aspect	Docker (Compose)	Kubernetes
Scope	One host	Cluster of N hosts
Scaling	Manual	Auto + declarative
Self-healing	✗	✓
Rolling updates	✗ (basic)	✓
Service discovery	Compose network	Built-in DNS
Learning curve	Low	Steep

3. Kubernetes Architecture

A cluster has two kinds of machines: **Control Plane** (the brain) and **Worker Nodes** (where your apps actually run).



Control Plane components

Component	Job
kube-apiserver	Front door. Everything talks to it via REST.
etcd	Distributed key-value store holding <i>all</i> cluster state.
kube-scheduler	Decides which Node a new Pod runs on.
controller-manager	Loops that reconcile actual state → desired state (e.g., ReplicaSet controller).
cloud-controller-manager	Glue between K8s and cloud APIs (load balancers, disks).

Node components

Component	Job
kubelet	Agent on each Node; makes sure containers in Pods are running.
kube-proxy	Maintains network rules for Service routing.
Container Runtime	Actually runs containers (containerd, CRI-O).

4. Core Objects: Pods, ReplicaSets, Deployments

● Pod

The **smallest deployable unit** in K8s. A Pod wraps 1 (usually) or more tightly-coupled containers that share network and storage.

▮ You almost never create Pods directly — you let a Deployment do it.

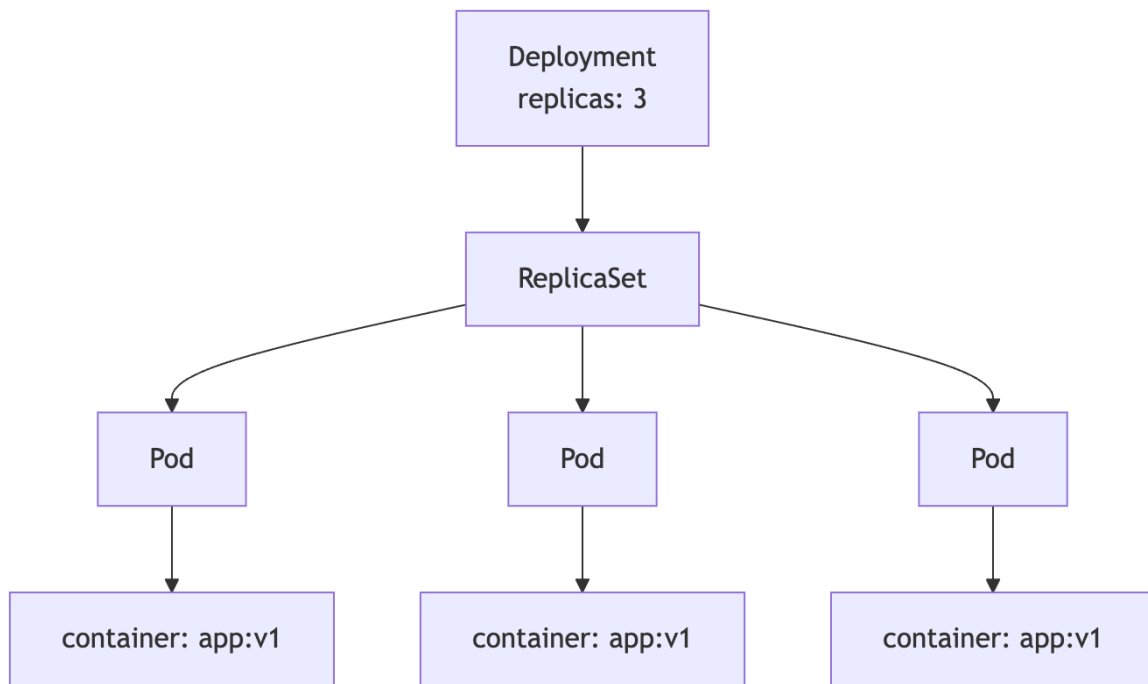
```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
```

🧬 ReplicaSet

Ensures **N copies of a Pod** are always running. If one dies, it spawns another.

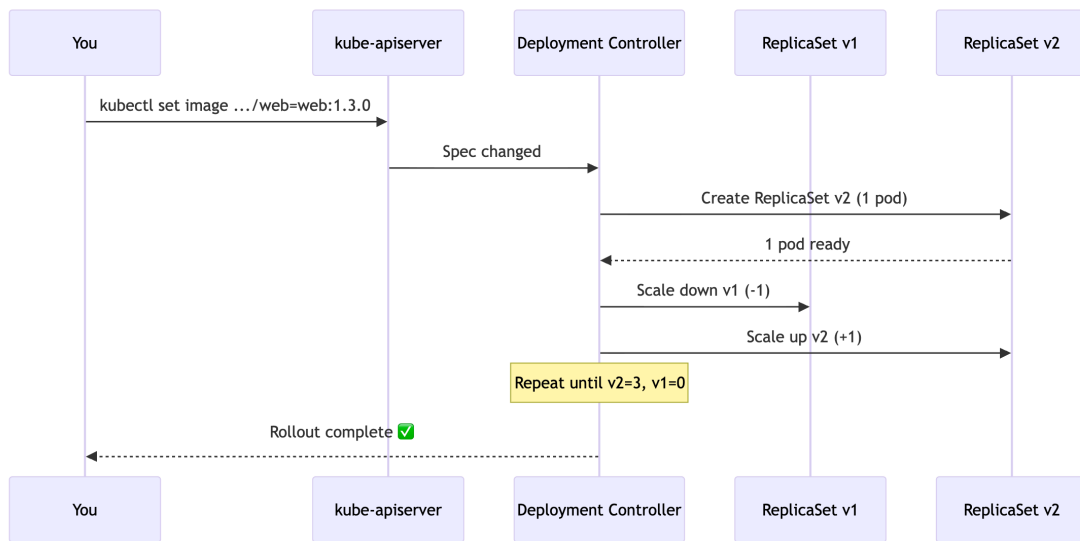
🚀 Deployment

The object you actually use 99% of the time. Manages a ReplicaSet and gives you **rolling updates + rollbacks**.



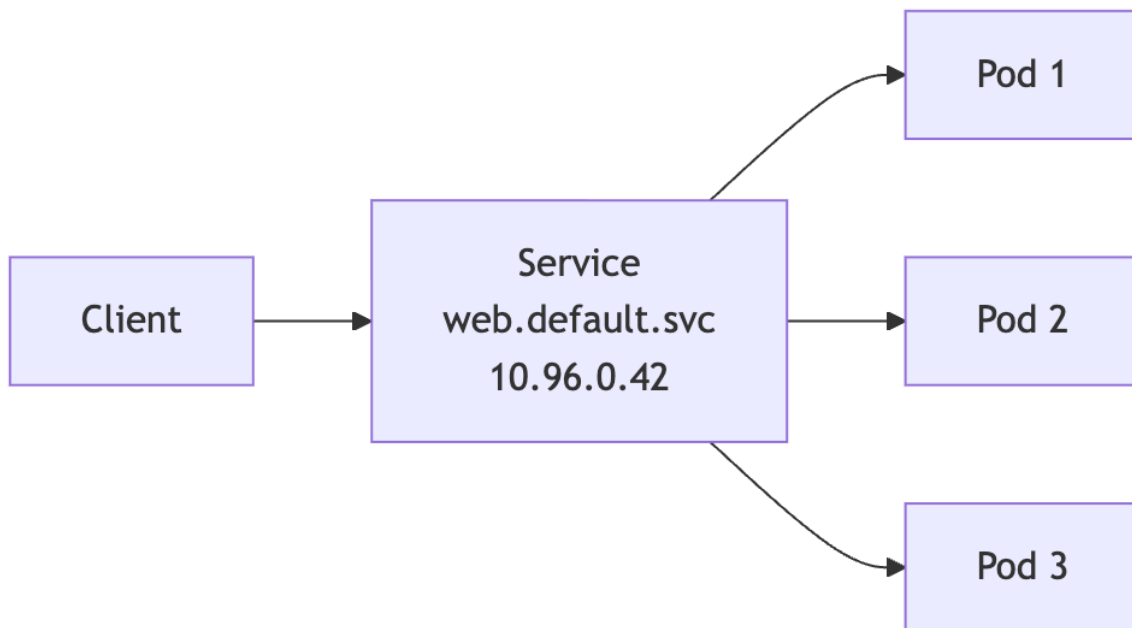
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: myrepo/web:1.2.0
          ports:
            - containerPort: 8080
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits:   { cpu: "500m", memory: "512Mi" }
```

Rolling update flow



5. Services & Networking

Pods are **ephemeral** — IPs change when they restart. A **Service** gives a stable virtual IP + DNS name in front of a set of Pods (chosen by label selector).



Service types

Type	Reachable from	Use case
ClusterIP (default)	Inside cluster only	Internal service-to-service
NodePort	<NodeIP>:<30000-32767>	Dev / simple external access
LoadBalancer	Public IP (cloud LB)	Production external traffic
ExternalName	DNS CNAME	Alias to an external service

```

apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: ClusterIP
  selector:
    app: web          # picks Pods with this label
  ports:
    - port: 80        # service port
      targetPort: 8080 # container port

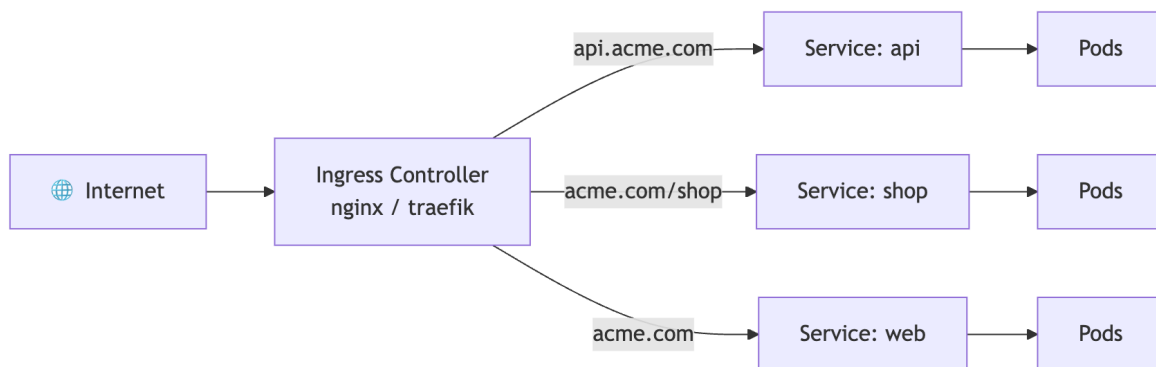
```

DNS

Every Service gets a DNS name: `<service>.<namespace>.svc.cluster.local`. From inside the same namespace you can just use `http://web`.

6. Ingress

A LoadBalancer per Service gets expensive fast. **Ingress** is a single HTTP(S) entry point that routes by **host** and **path** to many Services.



```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: acme
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt
spec:
  ingressClassName: nginx
  tls:
    - hosts: [acme.com, api.acme.com]
      secretName: acme-tls
  rules:
    - host: api.acme.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: api, port: { number: 80 } }
    - host: acme.com
      http:
        paths:
          - path: /shop
            pathType: Prefix
            backend:
              service: { name: shop, port: { number: 80 } }
          - path: /
            pathType: Prefix
            backend:
              service: { name: web, port: { number: 80 } }

```

You need an **Ingress Controller** installed in the cluster (nginx-ingress, Traefik, etc.) — the Ingress object alone does nothing.

7. ConfigMaps & Secrets

Never bake config or credentials into your image.

Object	For	Stored as
ConfigMap	Non-sensitive config (URLs, flags)	Plain text in etcd
Secret	Passwords, tokens, keys	Base64 in etcd (encrypt at rest!)

```

apiVersion: v1
kind: ConfigMap
metadata: { name: app-config }
data:
  LOG_LEVEL: info
  FEATURE_X: "true"
---
apiVersion: v1
kind: Secret
metadata: { name: db-creds }
type: Opaque
stringData:
  DB_PASSWORD: s3cret!

```

Consume them in a Pod:

```

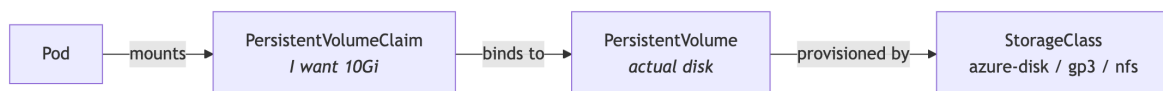
spec:
  containers:
    - name: app
      image: myapp:1.0
      envFrom:
        - configMapRef: { name: app-config }
        - secretRef:    { name: db-creds }

```

For real secrets in production, use **External Secrets Operator**, **SOPS**, or cloud secret managers (Azure Key Vault, AWS Secrets Manager).

8. Volumes & Persistent Storage

Container filesystems are **ephemeral**. For stateful data you need volumes.



Object	Role
Volume	Storage attached to a Pod (lifetime = Pod)
PersistentVolume (PV)	Cluster-wide piece of storage (lifetime = its own)
PersistentVolumeClaim (PVC)	A request for storage by a Pod
StorageClass	Recipe for dynamically provisioning PVs

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data }
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests: { storage: 10Gi }
  storageClassName: managed-csi

```


9. Namespaces, Labels & Selectors

Namespaces

Virtual clusters inside a cluster. Use them to separate `dev / staging / prod` or teams.

```
kubectl create namespace staging
kubectl get pods -n staging
```

Labels & Selectors

Labels are key/value tags on objects. **Selectors** query by them. This is how Services find Pods, Deployments manage ReplicaSets, etc.

```
metadata:
  labels:
    app: web
    tier: frontend
    env: prod
```

```
kubectl get pods -l app=web,env=prod
```

10. StatefulSets, DaemonSets, Jobs & CronJobs

Workload	Use case
Deployment	Stateless apps (web, APIs)
StatefulSet	Stateful apps needing stable identity + storage (Postgres, Kafka)
DaemonSet	One Pod per Node (log collectors, node-exporter)
Job	Run-to-completion task (DB migration)
CronJob	Scheduled Job (nightly backup)

```
apiVersion: batch/v1
kind: CronJob
metadata: { name: nightly-backup }
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: backup
              image: myorg/backup:1.0
```

11. Scheduling, Probes & Autoscaling

Probes — keep your Pods honest

Probe	Question it answers	If it fails
-------	---------------------	-------------

livenessProbe	Is the container alive?	Kill & restart it
readinessProbe	Is it ready for traffic?	Remove from Service endpoints
startupProbe	Has it finished booting?	Delay liveness checks

```
readinessProbe:
  httpGet: { path: /healthz, port: 8080 }
  initialDelaySeconds: 5
  periodSeconds: 10
```

Resource requests & limits

- **requests** → what the scheduler reserves for the Pod.
- **limits** → hard ceiling; CPU is throttled, memory triggers OOMKill.

Horizontal Pod Autoscaler (HPA)

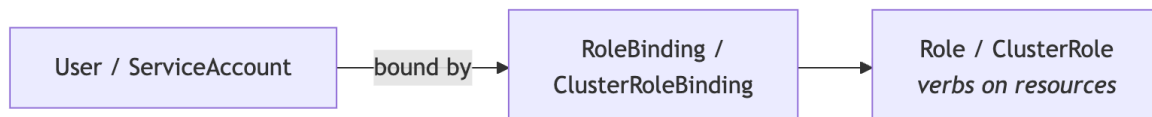
```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: web }
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
```

Scheduling controls

- **nodeSelector** — pin Pods to nodes with specific labels.
- **affinity / anti-affinity** — soft/hard rules ("spread my replicas across zones").
- **taints & tolerations** — nodes repel Pods unless the Pod tolerates the taint.

12. RBAC & Security

RBAC primitives



Object	Scope
Role	Namespace
ClusterRole	Whole cluster
RoleBinding	Grants a Role to a subject in a namespace

ClusterRoleBinding

Grants a ClusterRole cluster-wide

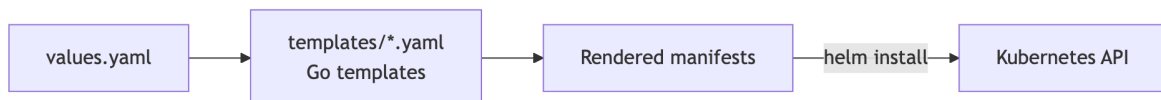
```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { namespace: dev, name: pod-reader }
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
```

Hardening checklist

- ✓ Run containers as **non-root** (`runAsNonRoot: true`).
- ✓ Drop all Linux capabilities; add back only what you need.
- ✓ Read-only root filesystem where possible.
- ✓ Use **NetworkPolicies** to restrict Pod-to-Pod traffic (default-deny).
- ✓ Enable **PodSecurityAdmission** (`restricted` profile).
- ✓ Scan images (Trivy, Gype) in CI.
- ✓ Rotate Secrets; encrypt etcd at rest.
- ✓ Pin image tags by **digest**, not `latest`.

13. Helm — The Package Manager

Writing 20 YAML files per app gets old. **Helm** packages them as a **chart** with templated values.



```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm install my-pg bitnami/postgresql --set auth.postgresPassword=secret
helm upgrade my-pg bitnami/postgresql -f values.yaml
helm rollback my-pg 1
helm uninstall my-pg
```

A chart structure:

```
mychart/
├─ Chart.yaml          # metadata
├─ values.yaml         # default config
└─ templates/
   ├─ deployment.yaml
   ├─ service.yaml
   └─ ingress.yaml
```

14. kubectl Cheat Sheet

```
# Cluster
kubectl cluster-info
kubectl get nodes -o wide
kubectl config get-contexts
kubectl config use-context my-cluster

# Apply / delete (declarative – preferred)
kubectl apply -f manifest.yaml
kubectl apply -k ./overlay/prod      # kustomize
kubectl delete -f manifest.yaml

# Inspect
kubectl get pods,svc,deploy -A
kubectl get pod web-abc -o yaml
kubectl describe pod web-abc
kubectl logs -f web-abc -c app      # follow, specific container
kubectl logs --previous web-abc     # crashed container's logs

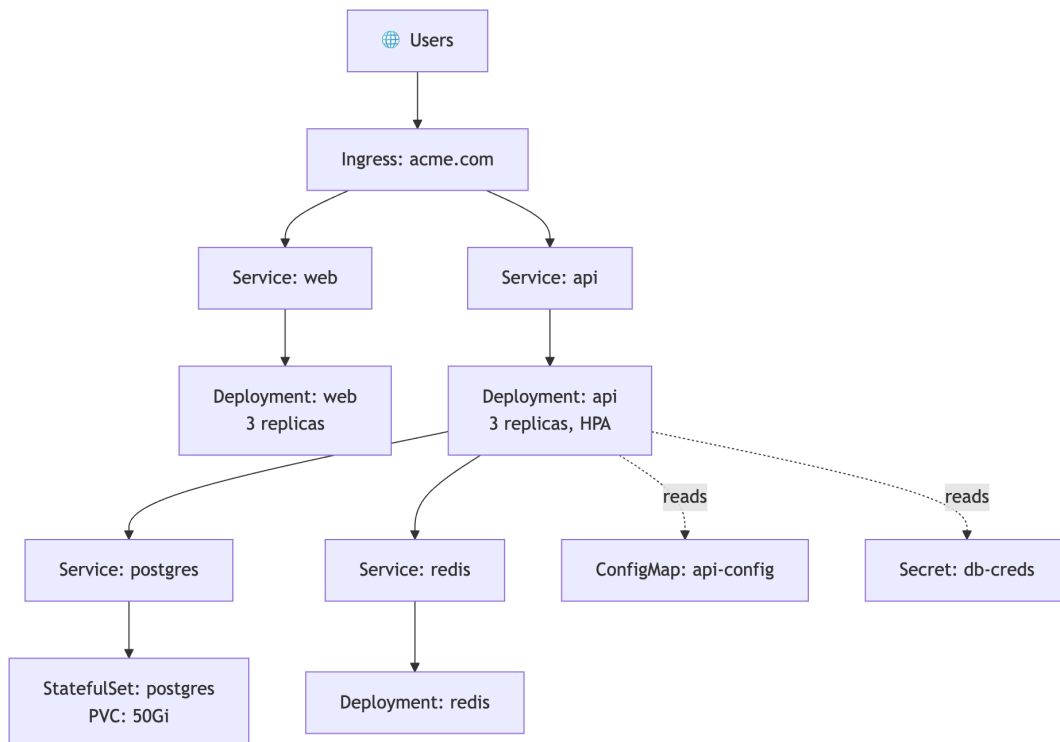
# Exec / debug
kubectl exec -it web-abc -- sh
kubectl port-forward svc/web 8080:80
kubectl run tmp --rm -it --image=busybox -- sh
kubectl debug node/node-1 -it --image=ubuntu

# Scale / rollout
kubectl scale deploy/web --replicas=5
kubectl set image deploy/web web=myrepo/web:1.3.0
kubectl rollout status deploy/web
kubectl rollout undo    deploy/web
kubectl rollout history deploy/web

# Resource usage (needs metrics-server)
kubectl top nodes
kubectl top pods -A
```

15. Real-World Example (fullstack app)

A typical web app on K8s:



Folder layout:

```

k8s/
├── namespace.yaml
├── configmap.yaml
├── secret.yaml
├── postgres-statefulset.yaml
├── redis-deployment.yaml
├── api-deployment.yaml
├── api-hpa.yaml
├── web-deployment.yaml
├── services.yaml
└── ingress.yaml
  
```

16. Troubleshooting

The Pod lifecycle (and what each status means)

Status	Meaning	Common cause
Pending	Not scheduled yet	No node has resources, or PVC not bound
ContainerCreating	Pulling image / mounting volumes	Slow registry, bad image name
Running	At least one container started	—
CrashLoopBackOff	Container keeps crashing	App error, bad command, failing probe
ImagePullBackOff	Can't pull image	Wrong tag, private registry without secret
OOMKilled	Hit memory limit	Raise <code>limits.memory</code> or fix leak

Error

/ Completed

Job-style Pod finished

—

Debug recipe

```
kubectl get pod <pod> -o wide
kubectl describe pod <pod>           # Events at the bottom – read these!
kubectl logs <pod> -c <container>
kubectl logs <pod> --previous        # for crashed containers
kubectl exec -it <pod> -- sh          # poke around inside
kubectl get events --sort-by=.lastTimestamp
```

Common gotchas

- **Service has no endpoints** → label selector doesn't match any Pod labels.
- **Probe failing right after deploy** → bump `initialDelaySeconds` or add a `startupProbe`.
- **Pod stuck Pending** → `kubectl describe` and look for *"0/3 nodes available: insufficient cpu"*.
- **ConfigMap change not picked up** → Pods don't auto-restart on ConfigMap edits; roll the Deployment (`kubectl rollout restart deploy/x`).
- **DNS resolution fails** → check CoreDNS pods in `kube-system`.

Mental Model — One Sentence Each

- **Cluster** = a fleet of machines pretending to be one big computer.
- **Pod** = the smallest unit K8s schedules (1+ containers sharing net/storage).
- **Deployment** = "keep N copies of this Pod running, and let me upgrade safely."
- **Service** = stable address in front of a set of Pods.
- **Ingress** = smart HTTP router from the internet to Services.
- **ConfigMap/Secret** = config and creds injected into Pods.
- **PVC** = "I need a disk this big, please."
- **Helm** = npm/apt for Kubernetes manifests.
- **kubectl** = your steering wheel.

Rule of thumb: describe the *desired state* in YAML, let Kubernetes' controllers do the reconciling. Stop thinking in commands; start thinking in manifests.